

AgriSmart AI: Comprehensive Project Blueprint

End-to-End Execution Plan, Tech Stack, & RAG Architecture for SIH-2026

1. Executive Summary & Core Objective

AgriSmart AI is an advanced agricultural advisory platform built for SIH-2026. It directly targets the primary evaluation hurdle: overcoming the *lab-to-field domain shift* by combining robust computer vision with localized Retrieval-Augmented Generation (RAG) and real-time environmental data streams.

2. Complete Technology Stack

2.1 Artificial Intelligence & Machine Learning

- **Computer Vision:** PyTorch & Ultralytics YOLOv8 (Stage 1 leaf localization & background cropping) + EfficientNet-B0 / ResNet-34 (Stage 2 disease classification).
- **Training Constraints (Dell G15 RTX 3050 4GB VRAM):** Optimized via batch=4 or 8, imgsz=640, and mixed-precision training (fp16=True) on Drive E.
- **Embeddings & RAG:** Hugging Face sentence-transformers (all-MiniLM-L6-v2) for dense vector generation of digitized textbooks.
- **Local LLM:** Ollama running quantized Llama-3-8B-Instruct (offloaded to 16GB DDR5 system RAM and CPU threads).

2.2 Backend, Database & Frontend

- **Backend API:** FastAPI (Python asynchronous routing for high-speed model execution).
- **Vector Store:** ChromaDB (offline local vector database stored in `vector_store/`).
- **Frontend UI:** Streamlit (Python) for instant rapid prototyping, camera upload, and interactive chat dashboards.

2.3 Bonus Modules & External Integrations

- **Weather Intelligence:** OpenWeatherMap API integration for live rain probability and temperature checks.
- **IoT Simulation:** Simulated sensor feeds for soil moisture, pH levels, and ambient temperature (fulfilling Bonus Module F).

3. End-to-End Architecture & RAG Pipeline Flow

The Execution Loop:

1. **Image Upload:** Farmer uploads a cluttered field photo via the frontend UI.
2. **Stage 1 (YOLO Leaf Detection):** Bounding box model isolates the plant leaf and crops out background noise.
3. **Stage 2 (Disease Classification):** EfficientNet evaluates the cropped leaf and outputs a strict string label (e.g., *Tomato_Early_blight*) and confidence score.
4. **RAG Retrieval:** The predicted disease class string triggers a semantic search inside the ChromaDB vector store, fetching exact treatment protocols from digitized agricultural textbooks.
5. **Environmental Context (Bonus):** Live weather and simulated soil sensor metrics are fetched and injected into the prompt.
6. **LLM Generation:** Local Llama-3 synthesizes the retrieved textbook context and sensor data into a grounded, plain-language advisory response.

4. Team Role Distribution

Module / Pillar	Owner	Key Responsibilities
CV & Core AI Training	Vatsal	FieldPlant dataset preparation, YOLOv8 & EfficientNet local training on Dell G15, writing <code>predict.py</code> interface.
Backend & RAG Engine	Vatsal	FastAPI setup, textbook OCR/parsing, ChromaDB embedding ingestion, and local Ollama LLM integration.
Frontend & UI/UX	Teammate A	Streamlit responsive dashboard, image upload component, and interactive chat assistant interface.
Weather & IoT Feed	Teammate B	OpenWeatherMap API integration and simulated soil moisture/pH sensor stream implementation.
Documentation & Video	Joint Effort	GitHub README.md, 1-page model report PDF (<code>`report/model_report.pdf`</code>), and 3-5 minute demo video.

5. Recommended Repository Structure (`agrismart-ai`)

```
agrismart-ai/
├── README.md                # Entry point, metrics, setup instructions (<10 mins run
                             # time)
├── requirements.txt          # Python dependencies (FastAPI, PyTorch, Langchain,
                             # etc.)
├── .env.example              # Environment configuration (API keys)
├── data/                    # Local storage on Drive E (~81GB free)
│   ├── raw_images/          # FieldPlant & PlantVillage datasets
│   └── textbooks_structured.json # Cleaned agronomy knowledge base
├── src/
│   ├── cv_pipeline/          # Stage 1 YOLO & Stage 2 Classifier training & inference
│   │   ├── train_yolo.py
│   │   ├── train_classifier.py
│   │   └── predict.py        # Mandatory core function: predict(image_path) ->
                             # class_label
│   ├── rag_pipeline/         # Knowledge retrieval engine
│   │   ├── ingest_vector_db.py # Chunking & embedding into ChromaDB
│   │   └── retriever.py       # Semantic search query logic
│   └── api/
│       └── main.py            # FastAPI backend server routing
├── app/
│   └── frontend_app.py        # Streamlit user interface
├── vector_store/              # Embedded vector database files
├── models/                    # Saved model weights (.pt / .pth)
└── report/                    # Mandatory 1-page model report PDF
```

6. SIH-2026 Compliance Checklist

- **Core Task:** Exposes a reproducible `predict(image_path)` function returning exact class strings and confidence scores.
- **Evaluation Metric:** Macro-F1 score and confusion matrix tracked and reported on validation splits.
- **GenAI Grounding:** RAG architecture strictly restricts LLM answers to verified textbooks, eliminating hallucinations.
- **Hardware Efficiency:** Fully functional offline deployment tailored for the Dell G15 hardware specs.